

Informed Search: Best First Search

Part-1

Introduction

The pursuit of artificial intelligence often begins with the fundamental challenge of problem-solving. Early AI research recognized that the ability to solve novel problems is a cornerstone of intelligence, leading to the development of general-purpose algorithms. These algorithms necessitated a general representation for problems, which culminated in the concept of a "search problem." Many real-world challenges can be framed as finding a path from a starting state to a goal state within a defined state space, implicitly defined by an expansion function. For decades, search has been considered so central to AI that some pioneers even equated AI with search itself, believing that mastering search could unlock solutions to all problems of intelligence.

However, the initial "uninformed" search algorithms, such as depth-first search, breadth-first search, iterative deepening search, beam search, and bidirectional search, while foundational, proved to be insufficient. They explore the search space blindly, without any directional guidance towards the goal. This lack of direction often leads to inefficient exploration, as the algorithms might investigate many irrelevant paths before stumbling upon a solution. To overcome this limitation, the field introduced "informed search," which aims to infuse search algorithms with intuition and guidance, making them smarter and more efficient in their quest for solutions. This lecture delves into the principles of informed search, focusing on how to add this crucial guidance to navigate complex problem spaces effectively.

Core Concepts

Term	Definition	Significance
Search Problem	A formal representation of a problem with states, actions, a start state, and a goal state, where the objective is to find a path from start to goal.	Provides a universal framework for applying search algorithms to diverse problems.
Uninformed Search	Search algorithms that explore the state space without any	Forms the baseline for search, but often inefficient

Term	Definition	Significance
	knowledge or guidance about the goal's location.	for large or complex problems.
Informed Search	Search algorithms that use additional problem-specific knowledge or "intuition" to guide their exploration towards the goal.	Significantly improves efficiency by prioritizing promising paths, making search practical for many AI problems.
Evaluation Function $f(n)$	A function that estimates the "goodness" or desirability of a node n , typically estimating the cost to reach the goal from n .	The core mechanism in best-first search for deciding which node to expand next, driving the search towards the goal.
Fringe (Open List)	A data structure (often a priority queue) holding all generated but not yet expanded nodes, ordered by their evaluation function $f(n)$.	Represents the set of candidate nodes for expansion, central to managing the search process.
Closed List (Explored List)	A data structure storing all nodes that have already been expanded, used to prevent revisiting and re-expanding nodes.	Essential for graph search to detect and avoid duplicate paths, ensuring efficiency and correctness.
Heuristic Function $h(n)$	An estimate of the cost from node n to the nearest goal state. It provides the "intuition" for informed search.	Guides the search by suggesting which nodes are likely closer to the goal, without performing a full path computation.
Path Cost $g(n)$	The actual cost of the path from the start state to node n .	Measures the cost incurred to reach a particular node, crucial for algorithms like Uniform Cost Search and A*.
	A general search algorithm that always expands the node from	A powerful paradigm that generalizes many search

Term	Definition	Significance
Best-First Search	the fringe with the lowest $f(n)$ value.	algorithms, both informed and uninformed, by varying $f(n)$.
Priority Queue	A data structure that efficiently retrieves the element with the highest or lowest priority (in this case, lowest $f(n)$).	The ideal data structure for implementing best-first search, allowing quick selection of the most promising node.

Detailed Analysis

1. The Transition from Uninformed to Informed Search

Traditional uninformed search algorithms, such as Depth-First Search (DFS), Breadth-First Search (BFS), Iterative Deepening Search (IDS), Beam Search, and Bidirectional Search, operate without any specific knowledge about the goal's location. They explore the search space systematically but blindly, often examining many paths that lead away from the solution. This can be highly inefficient, especially in large state spaces. The fundamental limitation is their lack of guidance; they don't prioritize nodes based on their potential to lead to the goal.

Informed search addresses this by incorporating "intuition" or problem-specific knowledge into the search process. Instead of blindly exploring, informed algorithms make educated guesses about which paths are more promising. This intuition, however, is not a perfect solution; it's a guess that can sometimes be accurate and sometimes misleading. The challenge lies in capturing this intuition within the algorithm to make smart decisions about which paths to pursue and which to temporarily ignore. The goal is to select nodes for expansion based on an evaluation function that estimates the cost to the goal, thereby guiding the search more directly.

2. The General Tree Search Paradigm

The general tree search paradigm provides a foundational structure for many search algorithms. It operates with a **fringe** (also known as a **frontier**), which is a collection of nodes that have been generated but not yet expanded.

The steps are as follows:

- **Initialization:** The root node (start state) is initially placed in the **fringe**.
- **Loop:** While the **fringe** is not empty:

- **Node Selection:** Remove a node from the `fringe` . In the context of best-first search, this node is chosen based on having the lowest `f(n)` value (evaluation function).
- **Goal Test:** Check if the removed node is the goal state. If it is, the search is successful, and the path to this node is the solution.
- **Expansion:** If the node is not the goal, generate all its successor nodes.
- **Fringe Update:** Add each successor node to the `fringe` . In a tree search, duplicate detection is not performed, meaning multiple paths to the same state are treated as distinct nodes.

This paradigm is highly flexible, and as we will see, even uninformed search algorithms can be viewed as special cases of this general structure by defining their `f(n)` appropriately.

3. The General Graph Search Paradigm

While tree search is useful, many problems involve state spaces where multiple paths can lead to the same state, forming a graph rather than a tree. To handle this efficiently and avoid redundant work, the general graph search paradigm introduces full duplicate detection. This requires two main data structures:

- **Open List (Fringe):** Similar to tree search, this holds generated but unexpanded nodes, typically ordered by their `f(n)` value.
- **Closed List (Explored List):** This list stores all nodes that have already been expanded. Its purpose is to remember visited states and prevent the algorithm from re-expanding them.

The graph search process modifies the tree search paradigm as follows:

- **Initialization:** The start node is added to the `open list` .
- **Loop:** While the `open list` is not empty:
- **Node Selection:** Remove the node with the lowest `f(n)` from the `open list` .
- **Goal Test:** Crucially, the goal test is performed when the node is removed from the `open list` , not when it's added. This is a common point of error for students.
- **Closed List Update:** If the node is not the goal, add it to the `closed list` .
- **Expansion:** Generate all successor nodes.
- **Fringe Update with Duplicate Detection:** For each successor:
 - Check if the successor is already in the `closed list` . If it is, ignore it (it has already been explored).
 - If it's not in the `closed list` , add it to the `open list` . (More advanced graph search algorithms might also check if a better path to a node already in the `open list` has been found, and update it).

This duplicate detection mechanism is vital for efficiency in graph-like state spaces, preventing infinite loops and redundant computations.

4. Best-First Search Algorithm

Best-First Search is a generic algorithm that embodies the principle of informed search. It uses an evaluation function $f(n)$ to determine the "best" node to expand next. The algorithm always selects the node from the fringe (or open list in graph search) that has the lowest $f(n)$ value. A low $f(n)$ value indicates a highly desirable node, suggesting it is likely to lead to the goal efficiently.

To implement best-first search efficiently, a priority queue (or a heap) is the ideal data structure for the fringe. This allows for quick retrieval of the node with the minimum $f(n)$ value. In contrast, Depth-First Search typically uses a stack (LIFO) and Breadth-First Search uses a queue (FIFO). Best-First Search generalizes these by allowing the $f(n)$ function to dictate the priority.

5. Illustrative Example: Romania Map Problem

Consider the problem of finding a path from Arad to Bucharest on a map of Romania. The state is the current city, actions are driving between cities, and the goal is Bucharest. For an informed search, we need additional information beyond just the map connections. This information comes in the form of a heuristic function.

A simple and effective heuristic for this problem is the straight-line distance (or "airplane distance") from any given city to Bucharest. If we are in a city like Sibiu, we can estimate its distance to Bucharest by drawing a straight line on the map. This estimate doesn't require computing the actual driving path; it's a quick "guess" about how close a city is to the goal. This $h(n)$ value (heuristic estimate from node n to the goal) provides the intuition: a city with a shorter straight-line distance to Bucharest is likely a better candidate to explore next. This guides the search towards the goal, much like a bird flying directly to its destination.

6. Uninformed Search as Special Cases of Best-First Search

One of the powerful aspects of the best-first search paradigm is its generality. Many uninformed search algorithms can be seen as specific instances of best-first search by defining their $f(n)$ function appropriately:

- **Breadth-First Search (BFS):** In BFS, nodes are expanded level by level. This can be achieved in a best-first search by setting $f(n)$ to be the depth of node n . When multiple nodes are in the fringe, BFS prioritizes those at a shallower depth. If $f(n) = \text{depth}(n)$, the priority queue will naturally expand nodes in increasing order of depth, mimicking BFS.
- **Uniform Cost Search (UCS):** UCS expands the node with the lowest cumulative cost from the start state. This is precisely what $g(n)$ represents

- the actual cost from the start to node n . Therefore, if we define $f(n) = g(n)$, a best-first search algorithm will behave exactly like Uniform Cost Search, always expanding the node that has been reached with the minimum path cost so far.

7. Key Functions in Informed Search

To fully understand and implement informed search algorithms like A*, it's crucial to distinguish between three core functions:

- **$f(n)$ (Evaluation Function):** This is the overall evaluation of a node n that best-first search uses to decide which node to expand next. It represents the estimated total cost of the path from the start, through n , to the goal.
- **$g(n)$ (Path Cost from Start):** This function represents the actual, known cost of the path from the initial start state to the current node n . It accumulates the costs of all edges traversed to reach n .
- **$h(n)$ (Heuristic Estimate to Goal):** This is the estimated cost from the current node n to the nearest goal state. It's the "intuition" or "guess" that guides the informed search.

These three functions are interconnected, with $f(n)$ often being a combination of $g(n)$ and $h(n)$, as seen in algorithms like A* search. Understanding their individual roles is fundamental to designing effective informed search strategies.

Key Takeaways

- [] Informed search algorithms use problem-specific knowledge or "intuition" to guide their exploration, unlike uninformed search.
- [] The core of informed search is an evaluation function $f(n)$ that estimates the desirability of a node, typically its cost to the goal.
- [] Best-First Search is a general paradigm that always expands the node from the fringe with the lowest $f(n)$ value.
- [] A priority queue or heap is the optimal data structure for implementing the fringe in best-first search.
- [] Graph search paradigms use a closed list (explored list) in addition to the open list (fringe) to perform full duplicate detection and avoid redundant work.
- [] The goal test in graph search should be performed when a node is removed from the fringe, not when it is added.

- [] Uninformed algorithms like Breadth-First Search ($f(n) = \text{depth}(n)$) and Uniform Cost Search ($f(n) = g(n)$) are special cases of best-first search.
- [] $g(n)$ is the actual cost from the start to node n , $h(n)$ is the estimated cost from node n to the goal, and $f(n)$ is the overall evaluation function.

Conclusion

The shift from uninformed to informed search marks a significant leap in the efficiency and practicality of AI problem-solving. By incorporating an evaluation function $f(n)$ and a heuristic function $h(n)$, algorithms like Best-First Search can intelligently navigate complex state spaces, prioritizing paths that are more likely to lead to a solution. This guidance transforms blind exploration into a more directed and intuitive process, mirroring how humans often approach problems.

The generality of the best-first search paradigm is particularly powerful, demonstrating how foundational uninformed methods can be re-contextualized as specific instances of a more intelligent approach. This understanding sets the stage for more advanced informed search algorithms, such as A* search, which further refine the balance between known path cost and estimated future cost to achieve optimal and efficient solutions. The ability to make educated guesses, even if imperfect, is a cornerstone of practical AI, enabling systems to tackle problems that would be intractable with brute-force methods alone.